SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech		Assignment Type: Lab		Program Name: B. Tech
Course Code	23CS002PC304	**Course Code**	23CS002PC304	
Year/Sem	III/II	**Year/Sem**	III/II	
Date and Day of Assignment	**Week5 – Wednesday**	**Date and Day of Assignment**	**Week5 – Wednesday**	
Name	M.Yashaswini	**Hall Ticket No**	2303A54049	

AssignmentNumber:10.3

Q.No.	Question
1	

Lab 9 – Code Review and Quality: Using AI to improve code quality and readability

Lab Objectives:

• To apply AI-based prompt engineering for code review and quality improvement.
• To analyze code for readability, logic, performance, and maintainability issues.
• To use Zero-shot, One-shot, and Few-shot prompting for improving code quality.
• To evaluate AI-generated improvements using standard coding practices.

Lab Outcomes (LOs): After completing this lab, students will be able to:

• Review and improve code quality using AI tools.
• Identify syntax, logic, and performance issues in code.
• Refactor code to improve readability and maintainability.
• Compare AI outputs generated using different prompting techniques.

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
    result = 1
    for i in range(1, n):
        result = result * i
    return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
   - Identify the logical bug in the code.
   - Explain why the bug occurs (e.g., off-by-one error).
   - Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

lab10.3-4049.py > ...

```python
# Prompt : Identify the logical error in the above code and correct it and explain why the error occurs and also provide a corrected version and also compare the both codes with comments
# The logical error in the above code is that the loop runs from 1 to n-1, which means it does not include n in the multiplication. This results in an incorrect factorial calculation for
# The corrected version of the code should run the loop from 1 to n (inclusive) to ensure that n is included in the multiplication.
# Corrected version:
Qodo: Test this function
def factorial(n):
    result = 1
    for i in range(1, n + 1):  # Loop runs from 1 to n (inclusive)
        result = result * i
    return result
n = int(input("Enter a number: "))
print("Factorial of", n, "is", factorial(n))
# Comparison of both codes:
# Original code:
# def factorial(n):
#     result = 1
#     for i in range(1,n):  # Loop runs from 1 to n-1
#         result = result * i
#     return result
# In the original code, if n is 5, the loop will multiply 1 * 2 * 3 * 4, which results in 24 instead of 120 (5! = 120).
# Corrected code:
# def factorial(n):
#     result = 1
#     for i in range(1, n + 1):  # Loop runs from 1 to n (inclusive)
#         result = result * i
#     return result
# In the corrected code, if n is 5, the loop will multiply 1 * 2 * 3 * 4 * 5, which results in the correct factorial value of 120.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS Python

```
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac> & c:\python314\python.exe c:/Users/srina/OneDrive/Documents/Desktop/aiac/lab10.3-4049.py
Enter a number: 5
Factorial of 5 is 24
Enter a number: 5
Factorial of 5 is 120
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac>
```

1. The manual code is missing the base case for when n is 0 or 1, which should return 1.
2. Additionally, the loop in the manual code should iterate up to n (inclusive) instead of n-1 to correctly calculate the factorial.
3. The AI-assisted code includes the base case and uses recursion to calculate the factorial, which is more efficient and easier to read.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

Instructions:

5. Use AI to:
    - Critique the function's readability, parameter naming, and lack of documentation.
    - Rewrite the function with:
        1. Descriptive function and parameter names.
        2. A complete docstring (description, parameters, return value, examples).
        3. Exception handling for division by zero.
        4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:
A well-documented, robust, and readable function that handles errors gracefully.

```python
#TASK 2:
#Manual code:
Qodo: Test this function
def calc(a,b,c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
        if b != 0:
            return a / b
        else:
            return "Error: Division by zero"
    else:
        return "Error: Invalid operation"
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
operation = input("Enter operation (add, sub, mul, div): ")
result = calc(a, b, operation)
print("Result:", result)
```

```python
#Rewrite the above function with descriptive function and parameters names with a complete docstring and exception handling for division by zero.
#AI code:
Qodo: Test this function
def calculate(num1, num2, operation):
    """
    Perform a basic arithmetic operation on two numbers.

    Parameters:
    num1 (float): The first number.
    num2 (float): The second number.
    operation (str): The operation to perform. It can be 'add', 'sub', 'mul', or 'div'.

    Returns:
    float: The result of the arithmetic operation.
    str: An error message if the operation is invalid or if division by zero occurs.
    """
    if operation == "add":
        return num1 + num2
    elif operation == "sub":
        return num1 - num2
    elif operation == "mul":
        return num1 * num2
    elif operation == "div":
        try:
            return num1 / num2
        except ZeroDivisionError:
            return "Error: Division by zero is not allowed."
    else:
        return "Error: Invalid operation. Please choose from 'add', 'sub', 'mul', or 'div'."
# Example usage:
number1 = float(input("Enter the first number: "))
number2 = float(input("Enter the second number: "))
operation = input("Enter operation (add, sub, mul, div): ")
result = calculate(number1, number2, operation)
print("Result:", result)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

PS C:\Users\srina\OneDrive\Documents\Desktop\aiac> & c:\python314\python.exe c:/Users/srina/OneDrive/Documents/Desktop/aiac/lab10.3-4049.py
Enter the first number: 6
Enter the second number: 6
Enter operation (add, sub, mul, div): mul
Result: 36.0
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac>
```

1. **The AI-assisted code is more readable and well-documented compared to the manual code.**
2. **It includes a docstring that explains the purpose of the function, its parameters, and its return value.**
3. **Additionally, the variable names in the AI-assisted code are more descriptive, making it easier to understand the functionality of the code at a glance.**

4. **The AI-assisted code also includes error handling for invalid operations and division by zero, which enhances its robustness.**

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
   - List all PEP8 violations.
   - Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

lab10.3-4049.py

lab10.3-4049.py > ...

```python
#TASK 3: Enforcing Coding Standards - PEP8 Compliance

# Original code with PEP8 violations:
Qodo: Test this function
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
# Generate the code List all PEP8 voilations in the above code and refactor the code and Write a short note on how automated AI reviews could streamlinecode reviews in large teams.

# PEP8 Violations identified:
# 1. Function name uses PascalCase instead of snake_case
# 2. Missing docstring
# 3. Logic error: Does not check if n <= 1 (not prime by definition)
# 4. Inefficient: Should check only up to sqrt(n)

# Test cases for original function:
print("Testing original function:")
print(f"Checkprime(2): {Checkprime(2)}")  # Expected: True, Gets: True
print(f"Checkprime(4): {Checkprime(4)}")  # Expected: False, Gets: False
print(f"Checkprime(1): {Checkprime(1)}")  # Expected: False, Gets: True (WRONG!)
print(f"Checkprime(0): {Checkprime(0)}")  # Expected: False, Gets: True (WRONG!)

# PEP8-compliant and corrected version:
Qodo: Test this function
def check_prime(n):
 '''
    Check if a number is prime.

    Parameters:
    n (int): The number to check for primality.

    Returns:
    bool: True if n is prime, False otherwise.'''
 if n < 2:
        return False
 for i in range(2, int(n ** 0.5) + 1):
     if n % i == 0:
         return False
 return True

# Test cases for corrected function:
print("\nTesting corrected function:")
print(f"check_prime(2): {check_prime(2)}")    # Expected: True
print(f"check_prime(4): {check_prime(4)}")    # Expected: False
print(f"check_prime(1): {check_prime(1)}")    # Expected: False
print(f"check_prime(0): {check_prime(0)}")    # Expected: False
print(f"check_prime(17): {check_prime(17)}")  # Expected: True
print(f"check_prime(20): {check_prime(20)}")  # Expected: False

# Note on AI-assisted code reviews for large teams:
# AI-powered code review tools streamline quality assurance by:
# 1. Automatically detecting PEP8 violations and style inconsistencies
# 2. Providing real-time suggestions before code submission
# 3. Reducing manual review time, allowing teams to focus on logic and architecture
# 4. Ensuring consistent coding standards across the entire codebase
# 5. Catching common bugs and inefficiencies (e.g., unnecessary iterations)
# 6. Generating documentation suggestions and format improvements
# 7. Scaling code reviews for large teams without additional overhead
```

```
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac> & c:\python314\python.exe
Testing original function:
Checkprime(2): True
Checkprime(4): False
Checkprime(1): True
Checkprime(0): True

Testing corrected function:
check_prime(2): True
check_prime(4): False
check_prime(1): False
check_prime(0): False
check_prime(17): True
check_prime(20): False
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac>
```

1. **The AI-assisted code has improved readability due to the use of descriptive variable names and a clear function name.**
2. **It also includes a docstring that explains the purpose of the function, its parameters, and its return value, which enhances documentation.**
3. **Additionally, the AI-assisted code adheres to coding standards by following a consistent naming convention (snake_case) and including error handling for edge cases (e.g., numbers less than or equal to 1). In contrast, the manual code lacks documentation and uses less descriptive variable names, making it less readable and harder to understand.**

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
   - Readability and naming.
   - Reusability and modularity.
   - Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
   a. Better naming and function purpose clarity.
   b. Input validation and type hints.
   c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

lab10.3-4049.py 4 ×

lab10.3-4049.py > ...

```python

# TASK 4: AI as a Code Reviewer in Real Projects

# Original code submitted by teammate:
Qodo: Test this function
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]

# Manual Code Review - Issues Identified:
# 1. Readability: Vague function name 'processData' and parameter 'd'
# 2. Reusability: Hardcoded multiplier (2) and operation (even filtering)
# 3. No input validation: Will crash on non-list input or non-numeric elements
# 4. No documentation: Purpose unclear
# 5. No type hints: Reduces code clarity and IDE support
# 6. Edge cases not handled: Empty lists, None values, non-integer types

# Refactored version with AI feedback applied:

Qodo: Test this function
def transform_even_numbers(numbers: List[Union[int, float]], multiplier: int = 2) -> List[Union[int, float]]:
    """
    Filter even numbers from a list and multiply them by a given factor.

    Parameters:
    numbers (List[Union[int, float]]): Input list containing numeric values.
    multiplier (int): The factor to multiply even numbers by (default: 2).

    Returns:
    List[Union[int, float]]: A list of transformed even numbers.

    Raises:
    TypeError: If input is not a list or contains non-numeric elements.
    """
    if not isinstance(numbers, list):
        raise TypeError(f"Expected list, got {type(numbers).__name__}")

    if not numbers:
        return []

    result = []
    for num in numbers:
        if isinstance(num, (int, float)) and num % 2 == 0:
            result.append(num * multiplier)
        elif not isinstance(num, (int, float)):
            raise TypeError(f"List contains non-numeric element: {num}")

    return result

# Test cases:
print("Test 1 - Normal case:", transform_even_numbers([1, 2, 3, 4, 5, 6]))
print("Test 2 - Empty list:", transform_even_numbers([]))
print("Test 3 - Custom multiplier:", transform_even_numbers([2, 4, 6], multiplier=5))
print("Test 4 - Floats:", transform_even_numbers([2.0, 3.5, 4.0]))
```

```
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac> & c:\python314\python.exe c:/Users/srina/OneDrive/Documents/Desktop/aiac/lab10.3-4049.py
Test 1 - Normal case: [4, 8, 12]
Test 2 - Empty list: []
Test 3 - Custom multiplier: [10, 20, 30]
Test 4 - Floats: [4.0, 8.0]
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac>
```

1. **The original code for the processData function was not following PEP8 style guidelines, and the function name was not descriptive of its purpose.**
2. **To fix this, we rename the function to `multiply_even_numbers` to better reflect its functionality. Additionally, we should remove the second argument from the example usage, as the function only takes one argument (the list of numbers). Finally, we should also add a docstring to the function to explain its purpose, parameters, and return value.**

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
```

```
        total += num ** 2
    return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:
   - Analyze time complexity.
   - Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
   - Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

lab10.3-4049.py × lab 7.5.py

lab10.3-4049.py > ...

```python
#Task5
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
numbers = range(1000000)
#Optimized High-Performance Code with numpy
import numpy as np
Qodo: Test this function
def sum_of_squares(numbers):
    """
    Calculate the sum of squares of a list of numbers.

    Args:
        numbers (iterable): An iterable of numerical values.
    Returns:
        float: The sum of squares of the input numbers.
    """
    return np.sum(np.square(numbers))
numbers = np.arange(1000000)
#Execution time comparision code
import time
# Original code execution time
start_time = time.time()
numbers = range(1000000)
print(sum_of_squares(numbers))
end_time = time.time()
print(f"Original code execution time: {end_time - start_time} seconds")
# Optimized code execution time
start_time = time.time()
numbers = np.arange(1000000)
print(sum_of_squares(numbers))
end_time = time.time()
print(f"Optimized code execution time: {end_time - start_time} seconds")
```

```
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac> & c:\python314\python.exe c
333332833333500000
Original code execution time: 0.07182717323303223 seconds
333332833333500000
Optimized code execution time: 0.00769805908203125 seconds
PS C:\Users\srina\OneDrive\Documents\Desktop\aiac>
```

1. **The original code for the sum_of_squares function uses a for loop to iterate through the list of numbers and calculate the sum of squares, which can be inefficient for large lists.**
2. **To optimize this, we can use the NumPy library, which provides efficient functions for array operations. We can use `np.square` to calculate the squares of the numbers and `np.sum` to calculate the total sum of those squares.**
3. **This approach is much faster, especially for large lists, as it takes advantage of NumPy's optimized C and Fortran code under the hood. Additionally, we should also add a docstring to the function to explain its purpose, parameters, and return value.**